

Python OOP

Er Piyush Pant

Mahendra Ratna Campus



Class and Objects



Class: class is a blueprint for creating objects. It defines **attributes** (data) and **methods** (functions) that the objects created from the class can use.



Object: An object is an instance of a class. It contains the actual values and can access the class methods and variables.

self and __init__

__init__ method in Python is a special method used to **initialize objects** of a class. It is automatically called **when a new object of the class is created**.

It's similar to constructor in other programming language.

self is a reference to the current object of a class.

It is used inside class methods to access the variables and methods of the specific object that is calling the method.

It must be the first parameter of instance methods, including the constructor (`__init__()`), though it is passed automatically when the method is called.

The self keyword helps each object maintain its own data and allows interaction with other functions and variables of the same object.

`__init__` Method (Constructor)

```
class Person:
    def __init__(self, name, age):
        # Initialize a new Person instance with a name and age.
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, my name is {self.name} and I am {self.age} years old."

# Example usage

if __name__ == "__main__":
    person1 = Person("Alice", 30)
    print(person1.greet())

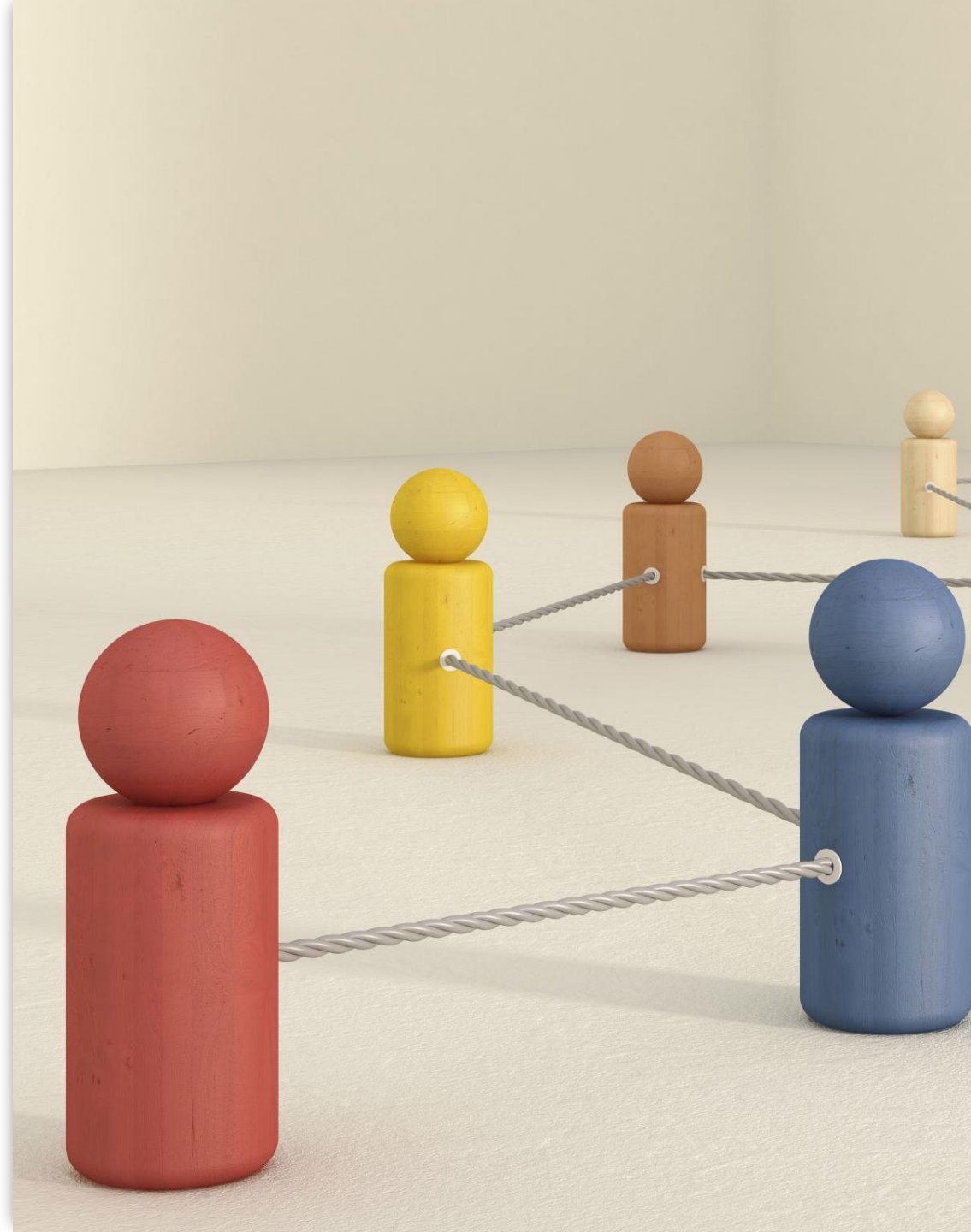
    person2 = Person("Bob", 25)
    print(person2.greet()) #
```


Inheritance

- Inheritance allows a class (child/derived class) to inherit the attributes and methods from another class (parent/base class).

Types of Inheritance in Python:

- Single Inheritance
- Multilevel Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance



Single Inheritance

Single Inheritance means that a **child class** inherits from **only one parent class**.

This allows the child class to reuse the properties and methods of the parent class.

```
class Animal:
    def speak(self):
        print("Animal speaks")
```

```
class Dog(Animal):
    def bark(self):
        print("Dog barks")
```

```
d = Dog()
d.speak() # Inherited from Animal
d.bark()  # Defined in Dog
```

Animal speaks
Dog barks

Multiple Inheritance

- Multiple inheritance is when a class inherits from more than one parent class.
- The child class gets attributes and methods from all the parent classes.

```
class Father:
    def skills(self):
        print("Gardening, Programming")

class Mother:
    def skills(self):
        print("Cooking, Art")

class Child(Father, Mother):
    def skills(self):
        Father.skills(self) # Call Father's skills
        Mother.skills(self) # Call Mother's skills
        print("Sports")

c = Child()
c.skills()
```

Multi Level Inheritance

- It is a type of inheritance where a class (child) inherits from another class (parent), and that parent class itself inherits from another class (grandparent).
- This creates a chain of inheritance where the child class can access properties and methods from all its ancestors.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def show_person(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Employee(Person):
    def __init__(self, name, age, employee_id):
        super().__init__(name, age)
        self.employee_id = employee_id

    def show_employee(self):
        print(f"Employee ID: {self.employee_id}")

class Manager(Employee):
    def __init__(self, name, age, employee_id, team_size):
        super().__init__(name, age, employee_id)
        self.team_size = team_size

    def show_manager(self):
        print(f"Manages a team of {self.team_size} people")

# Usage
mgr = Manager("Alice", 35, "E123", 10)
mgr.show_person()      # From Person class
mgr.show_employee()    # From Employee class
mgr.show_manager()     # From Manager class
```


Hierarchical Inheritance

- Hierarchical inheritance is when **multiple child classes inherit from a single parent class**. Each child class gets the features of the same parent class but can also have its own unique features.

```
class Animal:  
    def speak(self):  
        print("Animal speaks")
```

```
class Dog(Animal):  
    def bark(self):  
        print("Dog barks")
```

```
class Cat(Animal):  
    def meow(self):  
        print("Cat meows")
```

```
d = Dog()  
d.speak()  
d.bark()
```

```
c = Cat()  
c.speak()  
c.meow()
```

Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance. It mixes single, multiple, and multilevel inheritance to create a complex inheritance structure.

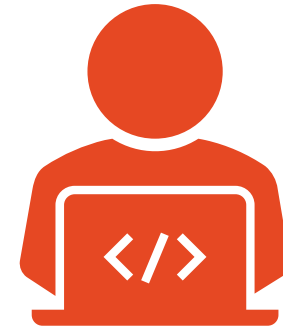
Example

- **Class A:** `Vehicle` (basic features like brand)
- **Class B:** `LandVehicle` (inherits from `Vehicle`, adds wheels info)
- **Class C:** `WaterVehicle` (inherits from `Vehicle`, adds water-specific info)
- **Class D:** `AmphibiousVehicle` (inherits from both `LandVehicle` and `WaterVehicle`)

Polymorphism



Polymorphism means "**many forms**".



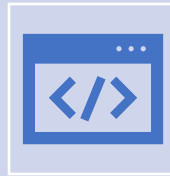
In Python, it allows the same method or function to behave differently depending on the object.

Polymorphism in functions

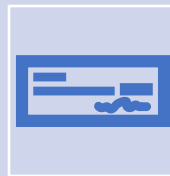
```
def add(a, b):  
    return a + b  
  
print(add(3, 4))           # Integer addition  
print(add("Hello, ", "World!")) # String concatenation  
print(add([1, 2], [3, 4])) # List concatenation
```

Data Hiding

- **Data hiding** means restricting access to the internal details of a class to protect data from being accidentally modified.
- It is a key part of **encapsulation** in object-oriented programming.



Python does **not** have true private variables like C++ or Java.



But it supports data hiding using **name mangling** with double underscores `__`.

Data Hiding

- **How it works**
- refixing a variable or method name with `__` makes it **hidden**(not directly accessible from outside the class).
- Internally, Python changes the name to `_ClassName__VariableName`.
- (self.__grade to self._Student__grade)

```
class Student:
    def __init__(self, name, grade):
        self.name = name           # Public attribute
        self.__grade = grade       # Private (hidden) attribute

    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade
        else:
            print("Invalid grade!")

    def get_grade(self):
        return self.__grade
```

Usage

```
s1 = Student("Piyush", 85)
```

```
print(s1.name)           # Output: Piyush
```

```
print(s1.get_grade())    # Output: 85
```

```
s1.set_grade(95)         # Modify safely using method
```

```
print(s1.get_grade())    # Output: 95
```

Try to access directly (will cause error)

```
# print(s1.__grade)      # ❌ Error
```

But can still access using name mangling (not recommended)

```
print(s1._Student__grade) # ⚠️ Output: 95
```

Types of Polymorphism

- **Duck Typing** – Using objects based on methods they have, not their type.
- **Operator Overloading** – Same operator works differently for different data types.
- **Method Overriding** – Child class redefines a parent class method.
- **Function Polymorphism** – Functions behave differently with different argument types.

Duck Typing

- **Duck Typing** is a concept in Python where the type of an object is not checked, only the presence of the required method or behavior is.
- It follows the idea:
- **"If it walks like a duck and quacks like a duck, it's treated like a duck."**
- In Python, as long as an object has the required method, it can be used, regardless of its actual class.

```
class Cat:  
    def speak(self):  
        print("Meow")
```

```
class Dog:  
    def speak(self):  
        print("Bark")
```

```
def make_sound(animal):  
    animal.speak() # No type check
```

```
make_sound(Cat()) # ✓ Works
```

```
make_sound(Dog()) # ✓ Works
```

Compared to C#

You must be of the **correct type** (interface or class) to be used."

✗ Will NOT work if Cat or Dog do not implement IAnimal.

Python: "Can you bark? Then go ahead."

C#: "Are you a Dog that implements IAnimal? If not, you're not allowed."

```
public interface IAnimal {  
    void Speak();  
}
```

```
public class Cat : IAnimal {  
    public void Speak() => Console.WriteLine("Meow");  
}
```

```
public class Dog : IAnimal {  
    public void Speak() => Console.WriteLine("Bark");  
}
```

```
void MakeSound(IAnimal animal) {  
    animal.Speak(); // Only works for IAnimal types  
}
```

Operator Overloading

Operator Overloading means giving **special meaning** to operators (like +, -, *) when they are used with user-defined objects.

Python lets you **define** how operators behave for your own classes by creating special methods like `__add__`, `__sub__`, etc.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"
```

```
p1 = Point(2, 3)
p2 = Point(4, 5)
print(p1 + p2)  # Output: (6, 8)
```

Some Common Operator that can be overloaded

Operator	Symbol	Special Method	Example Use
Addition	+	<code>__add__(self, other)</code>	<code>a + b</code>
Subtraction	-	<code>__sub__(self, other)</code>	<code>a - b</code>
Multiplication	*	<code>__mul__(self, other)</code>	<code>a * b</code>
Division	/	<code>__truediv__(self, other)</code>	<code>a / b</code>
Modulus	%	<code>__mod__(self, other)</code>	<code>a % b</code>
Equality	==	<code>__eq__(self, other)</code>	<code>a == b</code>
Greater Than	>	<code>__gt__(self, other)</code>	<code>a > b</code>
String Output	<code>str()</code>	<code>__str__(self)</code>	<code>print(a)</code>

Method Overriding

- **Method Overriding** is when a **child class** defines a method with the **same name** as a method in its **parent class**, replacing the parent's version.

Why to use

- To provide a **specific behavior** in the child class while keeping the same method name as in the parent class.

Example with Normal and Parent Call

```
class Employee:
    def salary(self):
        print("Base salary")

class Manager(Employee):
    def salary(self):
        print("Manager salary with bonus")

m = Manager()
m.salary() # Output: Manager salary with bonus
```

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):
        super().speak() # Call the parent class method
        print("Dog barks") # Child class behavior

# Create Dog object
d = Dog()
d.speak()
```

Function Polymorphism

- **Function Polymorphism** means the **same function name** can behave **differently** based on the **type of data** it receives.



Example

- `print(len("Python"))` # 6 → length of string
- `print(len([1, 2, 3]))` # 3 → number of list items
- `print(len({1: 'a', 2: 'b'}))` # 2 → number of dictionary keys

Since, python is dynamic type we cannot overload it with types of parameter like c# . We don't pass type .

User Defined Function Overloading

```
def add(*numbers):  
    return sum(numbers)  
  
print(add(2, 3))          # 5  
print(add(1, 2, 3, 4))    # 10
```

```
def add(a, b, c=0):  
    return a + b + c  
  
print(add(2, 3))          # 5  
print(add(2, 3, 4))       # 9
```

```
def show_info(*args):  
    if len(args) == 1:  
        print("Name:", args[0])  
    elif len(args) == 2:  
        print("Name:", args[0], "| Age:", args[1])  
    else:  
        print("Too many arguments")  
  
show_info("Alice")         # Name: Alice  
show_info("Bob", 25)       # Name: Bob | Age: 25
```